Security Analysis Report

AI Generated Authentication Library Implementation

For Cryptocurrency Wallet Applications

Classification Legend

✓ Secure
△ Partially Secure
× Insecure / Missing

Authors:

Aya Darawshi

Ibrahim Abomokh

AI tool used: Gemini 3 Flash Agent

Securing Digital Products and Services

Homework 1

Contents

Summary

#	Requirement	Classification
1	Password hashing algorithm	△ **Partially Secure**
2	Password policy enforcement	✓ **Secure**
3	Brute-force protection	✓ **Secure**
4	MFA implementation	△ **Partially Secure**
5	Password reset token generation	△ **Partially Secure**
6	Password reset validation	✓ **Secure**
7	Session generation method	△ **Partially Secure**
8	Token entropy	✓ **Secure**
9	Token expiration	✓ **Secure**
10	Session invalidation (logout)	✓ **Secure**
11	Session invalidation (password change)	△ **Partially Secure**
12	Cookie configuration	✓ **Secure**
13	Session fixation protection	✓ **Secure**
14	Privilege separation	× **Missing**
15	Key management	× **Insecure!**

Table 1: Security Requirements Compliance Summary

1 Password Hashing Algorithm & Configuration

1.1 Requirement Importance

Password hashing is the foundation of authentication security. Without hashing, a data breach will cause a disaster. Encryption is not secure against insiders. Hashing mitigates from insiders, but weak hashing or improperly configured algorithm allows attackers to crack passwords via rainbow tables, brute-force, or GPU attacks. Modern standards require memory-hard algorithms like Argon2id.

1.2 Locations in Code

File: `auth_lib/core/security.py` (lines 10–27):

```python
# Argon2id setup
ph = PasswordHasher(
    memory_cost=settings.ARGON2_MEMORY_COST,
    time_cost=settings.ARGON2_TIME_COST,
    parallelism=settings.ARGON2_PARALLELISM,
    type=Type.ID
)

def hash_password(password: str) -> str:
    """Hashes a password using Argon2id."""
    return ph.hash(password)

def verify_password(hashed_password: str, password: str) -> bool:
    """Verifies a password against an Argon2id hash."""
    try:
        return ph.verify(hashed_password, password)
    except VerifyMismatchError:
        return False
```

File: `auth_lib/core/config.py` (lines 14–17):

```python
# Argon2id Settings
ARGON2_MEMORY_COST: int = 64 * 1024  # 64 MB in KB
ARGON2_TIME_COST: int = 3
ARGON2_PARALLELISM: int = 2
```

1.3 Security Analysis

Strengths: Uses `Argon2id` algorithm—the `id` variant of the Argon2 algorithm provides resistance against side-channel attacks and GPU attacks. The Argon2 Python library securely handles salts automatically and transparently (preventing rainbow-table attacks). Configured parameters exceed OWASP minimum recommendations (19MB, 2 iterations).

Weaknesses:

Incomplete Exception Handling: The `argon2` library can raise:

- `VerifyMismatchError`
- `InvalidHash`

Only `VerifyMismatchError` is handled. If `InvalidHash` raises, this would cause an unhandled exception and potentially leak error details.

Attack Scenario: Attacker corrupts a stored password hash of a specific victim (via SQL injection or insider access) and modifies it to invalid format, causing `ph.verify()` to throw `InvalidHash` for any normal login attempt.

Impact: Crash the service, expose stack traces, DoS.

Fix: Handle both exceptions.

No Rehash Support: If Argon2 parameters are later updated to stay ahead of hardware advances, existing old hashes won't be updated automatically and therefore remain weak.

Attack Scenario: Argon2 parameters updated and old user hashes remain weak. Attacker steals database and cracks old hashes using SOTA hardware.

Impact: Legacy passwords exposed.

Fix: Rehash passwords whenever Argon2 parameters are updated. This can be done whenever a user logs in (since that is the only time we have the cleartext password). The Argon2 Python library provides a method called `check_needs_rehash` specifically for this purpose.

1.4 Classification

△ Partially Secure

2 Password Policy Enforcement (Length/Complexity)

2.1 Requirement Importance

Weak passwords are vulnerable to password spraying and credential stuffing. Therefore, it's crucial for high-risk apps to enforce password policy. However, password policy raises a trade-off between security and usability. Additionally, enforcement of complex passwords may raise security concerns because it encourages users to choose predictable patterns in passwords (e.g., adding '!' to the end of the password to satisfy the policy). On the other hand, the absence of policy allows for weak passwords.

2.2 Location in Code

File: `auth_lib/schemas/schemas.py` (lines 4–6):

```
class UserCreate(BaseModel):
    email: EmailStr
    password: str = Field(..., min_length=14, max_length=128)
```

File: `auth_lib/services/auth_service.py` (lines 17–23, 129–133):

```
def register_user(self, email: str, password: str) -> User:
    # 1. Complexity check
    if len(password) < 14 or len(password) > 128:
        raise ValueError("Password must be between 14 and 128
            characters.")

    # 2. HIBP check
    if ExternalSecurityService.is_password_pwned(password):
        raise ValueError("This password has been found in a data
            breach.")
    # ...
```

File: `external_api_service.py` (lines 9–30):

```
def is_password_pwned(password: str) -> bool:
    """Checks if a password has been leaked using HIBP k-anonymity
       API."""
    sha1_hash = hashlib.sha1(password.encode()).hexdigest().upper()
    prefix = sha1_hash[:5]
    suffix = sha1_hash[5:]

    try:
        url = f"https://api.pwnedpasswords.com/range/{prefix}"
        response = requests.get(url, timeout=5)
        response.raise_for_status()

        hashes = (line.split(':') for line in
            response.text.splitlines())
        for h, count in hashes:
            if h == suffix:
                return int(count) > 0
        return False
    except Exception as e:
        logger.error(f"HIBP API error: {e}")
        return False  # Fail-safe
```

2.3 Security Analysis

Strengths:

- 14-character minimum exceeds best practice recommendations (8 chars)
- 128-character maximum prevents DoS via long password hashing
- HIBP (Have I Been Pwned) integration protects against breached passwords
- K-anonymity ensures the actual password is never sent to HIBP
- No complexity requirements (uppercase, numbers, symbols)—eliminating predictable patterns in passwords

Weaknesses:

Missing Failure Handling: When the HIBP API fails, the implementation returns `False` (fail-open), allowing potentially breached passwords. The README documents this as a UX trade-off.

> Attack Scenario: Assuming the attacker has access to the hashed passwords in the database (SQL injection, insider, etc.):
>
> 1. Attacker performs a targeted DDoS on the HIBP API endpoint, or exploits network instability between the server and HIBP
> 2. During this window, new clients may register accounts with breached passwords
> 3. Attacker performs dictionary attack hoping to crack weak passwords registered during the DDoS attack
>
> *However, performing such an attack seems impractical.*
>
> Impact: Accounts created with breached passwords are vulnerable to credential stuffing and password spraying attacks.
>
> Fix: Implement failure handling—reject registration if HIBP is unreachable. Alternatively, allow registration but force password change on first login after HIBP becomes available.

2.4 Classification

✓ **Secure**

3 Brute-Force / Rate Limiting Protection

3.1 Requirement Importance

Without rate limiting, attackers can attempt millions of password combinations, or send too many requests to cause DoS. Rate limiting prevents credential stuffing, password spraying, brute-force attacks, and DoS attacks against authentication endpoints.

3.2 Location in Code

File: `auth_lib/services/rate_limit_service.py` (lines 1–201):

```
class RateLimitService:
    def __init__(self, db: DBSession):
        self.db = db

    def is_rate_limited(self, key: str, capacity: int, refill_rate: float) -> bool:
        """Token Bucket Rate Limiter (SQLite-backed)."""
        bucket_key = f"rl:bucket:{key}"
        now = datetime.now(timezone.utc)

        bucket = self.db.query(RateLimitBucket).filter(
            RateLimitBucket.bucket_key == bucket_key
        ).first()

        if not bucket:
            bucket = RateLimitBucket(bucket_key=bucket_key, tokens=capacity - 1, last_refill=now)
            self.db.add(bucket)
            self.db.commit()
            return False

        # Calculate token refill
        elapsed = (now - bucket.last_refill.replace(tzinfo=timezone.utc)).total_seconds()
        new_tokens = min(capacity, bucket.tokens + int(elapsed * refill_rate))

        if new_tokens >= 1:
            bucket.tokens = new_tokens - 1
            bucket.last_refill = now
            self.db.commit()
            return False
        return True

    def increment_failed_login(self, identifier: str) -> int:
        """Increments failed login counter. Counters older than 24h are reset."""
        # ... implementation ...

    def is_account_locked(self, identifier: str) -> bool:
        """Checks if account/IP is locked. Cleans up expired lockouts."""
        # ... implementation ...

    def set_lockout(self, identifier: str, duration_minutes: int):
        """Sets a temporary lockout."""
```

```
41          # ... implementation ...
```

File: `auth_lib/api/main.py` (lines 43–60):

```
# 1. Rate Limiting check
ip = request.client.host
if rate_limit.is_account_locked(login_data.email) or
    rate_limit.is_account_locked(ip):
     raise HTTPException(status_code=429, detail="Account or IP
        locked.")

# ... authentication ...

if count >= settings.MAX_FAILED_LOGIN_ATTEMPTS:
     lockout_idx = min(count - settings.MAX_FAILED_LOGIN_ATTEMPTS,
                       len(settings.LOCKOUT_DURATION_MINUTES) - 1)
     duration = settings.LOCKOUT_DURATION_MINUTES[lockout_idx]
     rate_limit.set_lockout(login_data.email, duration)
     rate_limit.set_lockout(ip, duration)
```

Configuration (`config.py`):

```
MAX_FAILED_LOGIN_ATTEMPTS: int = 5
LOCKOUT_DURATION_MINUTES: List[int] = [5, 15, 30, 60, 1440]  #
    Progressive
```

3.3 Security Analysis

Strengths:

- Both IP AND account are rate-limited, mitigating both distributed attacks (across multiple accounts) and targeted attacks (against single accounts)
- Progressive lockouts prevent sustained attacks (5m → 24h)
- Token bucket algorithm implemented, allowing legitimate users who mistype their password while blocking automated attacks
- 24-hour automatic expiry on failed attempt counters prevents permanent account lockout DoS attacks (maintains availability)

Weaknesses:

IP Reputation and Geographic Anomaly: `get_ip_reputation()` and `check_geographic_anomaly()` are implemented in `external_api_service.py` (lines 33–63) but never called anywhere in the authentication flow! The absence of checking geographic anomaly may prevent identifying credential stuffing. While the absence of checking IP reputation will make it easier for an attacker to complete their attack.

3.4 Classification

✓ **Secure**

4 Multi-Factor Authentication Implementation

4.1 Requirement Importance

MFA provides defense-in-depth: even if passwords are compromised, attackers cannot access accounts without the second factor. TOTP is a widely-supported MFA method.

4.2 Location in Code

File: `auth_lib/services/mfa_service.py` (lines 1–31):

```python
class MFAService:
    @staticmethod
    def generate_new_totp_secret() -> str:
        """Generates a new base32 TOTP secret."""
        return pyotp.random_base32()

    @staticmethod
    def get_totp_instance(secret: str) -> pyotp.TOTP:
        return pyotp.TOTP(
            secret,
            interval=settings.TOTP_STEP,    # 30 seconds
            digits=settings.TOTP_DIGITS     # 6 digits
        )

    def verify_totp(self, secret_encrypted: str, code: str) -> bool:
        secret = encryption_manager.decrypt(secret_encrypted)
        totp = self.get_totp_instance(secret)
        return totp.verify(code, valid_window=1)  # 1 step tolerance

    def encrypt_secret(self, secret: str) -> str:
        return encryption_manager.encrypt(secret)
```

File: `auth_lib/api/main.py` (lines 63–66):

```python
# 3. MFA Verification (Required by default)
if not
    auth_service.mfa_service.verify_totp(user.mfa.totp_secret_encrypted,
    login_data.totp_code):
     rate_limit.increment_failed_login(login_data.email)
     raise HTTPException(status_code=401, detail="Invalid MFA code")
```

4.3 Security Analysis

Strengths:

- MFA is mandatory for all accounts, preventing password-only attacks
- TOTP secrets encrypted at rest with Fernet, protecting secrets if database is compromised
- Uses `pyotp.random_base32()` which uses OS CSPRNG, ensuring unpredictable secrets
- Standard 30-second time step
- MFA required for password reset, preventing email-only account takeover

Weaknesses:

Missing features for comprehensive sensitive action protection and high-risk scenario protection:

No High-Risk Scenario Detection: `check_geographic_anomaly()` is implemented but not integrated. Logins from new devices/locations don't trigger additional verification.

Attack Scenario: Attacker with stolen credentials logs in from different country. No additional challenge is presented.

Impact: Account takeover from unusual locations goes undetected.

Fix: Integrate geographic anomaly detection and require step-up authentication for suspicious logins.

No Backup Codes: If user loses authenticator device, account recovery is impossible without admin intervention.

No Rate Limiting on TOTP Attempts: Failed TOTP attempts increment the general login counter, but a user who passes password check can potentially attempt many TOTP codes.

Attack Scenario: Attacker knows password, attempts to brute-force 6-digit TOTP (1,000,000 combinations).

Impact: With 30-second window and no TOTP-specific rate limit, brute-force is theoretically possible.

Fix: Add separate TOTP attempt counter with strict lockout (e.g., 3 failed TOTP attempts = lockout).

4.4 Classification

△ Partially Secure

5 Password Reset Token Generation

5.1 Requirement Importance

Password reset tokens must be unpredictable and cryptographically random. Weak tokens allow attackers to predict or brute-force reset links, enabling unauthorized password changes. Additionally, the reset flow must not leak information about which accounts exist (user enumeration).

5.2 Location in Code

File: `auth_lib/services/auth_service.py` (lines 96–110):

```python
def initiate_password_reset(self, email: str):
    user = self.db.query(User).filter(User.email == email).first()
    if not user:
        return  # Don't reveal if user exists

    token = generate_secure_token(16)  # 128 bits
    r_token = VerificationToken(
        user_id=user.id,
        token_hash=hash_token(token),
        expires_at=datetime.now(timezone.utc) + timedelta(minutes=5),
        type='password_reset'
    )
    self.db.add(r_token)
    self.db.commit()
    self.email_service.send_password_reset_email(email, token)
```

File: `auth_lib/core/security.py` (lines 29–35):

```python
def generate_secure_token(nbytes: int = 16) -> str:
    """Generates a CSPRNG cryptographically-secure random token."""
    return secrets.token_urlsafe(nbytes)

def hash_token(token: str) -> str:
    """Hashes a token for secure storage."""
    return hashlib.sha256(token.encode()).hexdigest()
```

5.3 Security Analysis

Strengths:

- Uses Python's `secrets` module (OS-level CSPRNG) ensuring tokens are cryptographically random
- 128-bit tokens (2^{128} combinations) are infeasible to brute-force
- Tokens stored as SHA256 hash, so database compromise doesn't reveal valid tokens

Weaknesses:

No Rate Limiting on Reset Requests: The `/password-reset/request` endpoint has no rate limiting. An attacker can spam reset requests for any email address. Additionally, notice that in the implementation of `initiate_password_reset`, if the user does not exist the function does not generate a token and hash it. This may enable timing-based side-channel attack.

> Attack Scenario 1: Attacker floods victim's inbox with password reset emails (email bombing).

Attack Scenario 2: Attacker attempts to enumerate valid accounts by timing response differences.

Impact: User harassment, potential email provider rate limiting (scenario 1); Timing-based user enumeration (scenario 2).

Fix: Always complete computing `initiate_password_reset` even if the user email doesn't exist (reducing timing differences). Add rate limiting to the reset request endpoint for emails (preventing scenario 1) and for IPs (preventing scenario 2).

5.4 Classification

△ Partially Secure

6 Password Reset Validation & Expiration

6.1 Requirement Importance

Reset tokens must expire quickly and be single-use. Long-lived or reusable tokens expand the attack window and allow token replay attacks. Additionally, the reset process should be protected against brute-force attempts on the token itself.

6.2 Location in Code

File: `auth_lib/services/auth_service.py` (lines 112–139):

```
def complete_password_reset(self, token: str, new_password: str,
    totp_code: str):
    token_hash = hash_token(token)
    r_token = self.db.query(VerificationToken).filter(
        VerificationToken.token_hash == token_hash,
        VerificationToken.is_used == False,
        VerificationToken.type == 'password_reset'
    ).first()

    if not r_token or
       r_token.expires_at.replace(tzinfo=timezone.utc) <
       datetime.now(timezone.utc):
        raise ValueError("Invalid or expired reset token.")

    user = self.db.query(User).get(r_token.user_id)

    # Require MFA for password reset
    if not
       self.mfa_service.verify_totp(user.mfa.totp_secret_encrypted,
       totp_code):
        raise ValueError("Invalid MFA code.")

    if len(new_password) < 14:
        raise ValueError("Password too short.")

    if ExternalSecurityService.is_password_pwned(new_password):
        raise ValueError("Password is pwned.")

    user.hashed_password = hash_password(new_password)
    user.last_password_change = datetime.now(timezone.utc)
    r_token.is_used = True  # Mark as used
    self.db.commit()
    return user
```

6.3 Security Analysis

Strengths:

- 5-minute expiration provides a short attack window
- Single-use enforcement via `is_used` flag prevents token replay
- MFA required for password reset provides defense-in-depth (email compromise alone is insufficient)
- New password validated against HIBP
- All sessions invalidated after successful reset, ejecting any attacker with active sessions
- `last_password_change` timestamp updated for audit purposes

Weaknesses:

Token Not Invalidated After Failed MFA Attempts: If an attacker has the reset token but not MFA, they can attempt TOTP codes indefinitely within the 5-minute window without the token being invalidated.

> Attack Scenario: Attacker intercepts reset email (e.g., compromised email server logs). They have the token but not MFA. They can attempt ∼1,000,000 TOTP combinations. At 100 requests/second, they could attempt 30,000 codes in 5 minutes (3% of total space per window).
>
> Impact: Statistically unlikely to succeed for a single token, but repeated attempts across multiple reset tokens could eventually succeed.
>
> Fix: Add attempt counter to reset tokens. Invalidate token after 3–5 failed MFA attempts.

No Cleanup of Expired Tokens: Expired and used tokens remain in the database indefinitely. Over time, this causes database inflation.

6.4 Classification

✓ Secure

7 Session/Token Generation Method

7.1 Requirement Importance

Session IDs must be cryptographically random (therefore unpredictable). Predictable session IDs enable session hijacking attacks. The session must also be bound to contextual information (device, IP) to detect theft.

7.2 Location in Code

File: `auth_lib/core/security.py` (lines 58–65):

```python
def generate_session_id() -> str:
    """Generates a 128-bit session ID, Base64 URL-encoded."""
    return secrets.token_urlsafe(16)  # 16 bytes = 128 bits

def generate_device_fingerprint(user_agent: str, ip_subnet: str,
    lang: str) -> str:
    """Creates a hash of device characteristics for session
       binding."""
    data = f"{user_agent}|{ip_subnet}|{lang}"
    return hashlib.sha256(data.encode()).hexdigest()[:32]
```

File: `auth_lib/services/session_service.py` (lines 14–31):

```python
def create_session(self, user_id: int, device_fingerprint: str =
    None) -> Tuple[str, Session]:
    """Creates a new session for a user."""
    session_id = generate_session_id()
    session_id_hash = hash_token(session_id)

    expires_at = datetime.now(timezone.utc) +
       timedelta(minutes=settings.SESSION_ABSOLUTE_TIMEOUT)

    db_session = Session(
        user_id=user_id,
        session_id_hash=session_id_hash,
        expires_at=expires_at,
        device_fingerprint=device_fingerprint
    )
    self.db.add(db_session)
    self.db.commit()
    return session_id, db_session
```

7.3 Security Analysis

Strengths:

- Uses `secrets.token_urlsafe()` ensuring session IDs are cryptographically random
- 128-bit session IDs exceed OWASP minimum (64 bits), making brute-force infeasible
- Session IDs stored as SHA256 hash, so database compromise doesn't reveal valid session tokens
- Device fingerprint binding adds contextual security, enabling detection of session theft from different device/network

Weaknesses:

Fingerprint Uses IP Subnet, Not Full IP: The fingerprint uses only a prefix of the IP (`ip_subnet = ".".join(ip_parts[:3])`). This allows session use across the same /24 subnet.

> Attack Scenario: Attacker on the same network (e.g., corporate LAN, public WiFi) steals session cookie. Since they share the same /24 subnet, the fingerprint matches.
>
> Impact: Less effective device binding.
>
> Fix: Use full IP address in fingerprint.

7.4 Classification

△ Partially Secure

8 Token Entropy & Randomness Quality

8.1 Requirement Importance

Insufficient entropy makes tokens predictable. Attackers can enumerate weak tokens through brute-force, or exploit patterns in pseudo-random generators. All security-critical tokens (session IDs, reset tokens, verification tokens, etc.) must use cryptographically secure random number generators (CSPRNG).

8.2 Location in Code

File: `auth_lib/core/security.py` (lines 29–35, 58–61):

```python
import secrets

def generate_secure_token(nbytes: int = 16) -> str:
    """Generates a CSPRNG cryptographically-secure random token."""
    return secrets.token_urlsafe(nbytes)

def generate_session_id() -> str:
    """Generates a 128-bit session ID, Base64 URL-encoded."""
    return secrets.token_urlsafe(16)
```

File: `auth_lib/services/mfa_service.py` (lines 7–9):

```python
@staticmethod
def generate_new_totp_secret() -> str:
    """Generates a new base32 TOTP secret."""
    return pyotp.random_base32()  # Uses secrets.SystemRandom
        internally
```

8.3 Security Analysis

Strengths:

- All tokens use Python's `secrets` module, which sources from OS-level CSPRNG designed to be cryptographically secure and resistant to prediction
- 128-bit entropy for all tokens provides infeasibility against brute-force
- URL-safe encoding makes it transport-safe
- TOTP secrets use `pyotp.random_base32()` which internally uses `secrets.SystemRandom`, ensuring MFA secrets are also unpredictable

Weaknesses:

None identified.

8.4 Classification

✓ **Secure**

9 Token Expiration Mechanism

9.1 Requirement Importance

Tokens without expiration remain valid indefinitely, increasing the window for theft and replay attacks. Proper expiration with both absolute and idle timeouts limits attacker opportunities even if a session token is stolen.

9.2 Location in Code

File: `auth_lib/core/config.py` (lines 19–23):

```
# Session Settings
SESSION_ABSOLUTE_TIMEOUT: int = 120    # 2 hours in minutes
SESSION_IDLE_TIMEOUT: int = 10         # 10 minutes
SESSION_ROTATION_INTERVAL: int = 150   # 2.5 minutes in seconds
```

File: `auth_lib/services/session_service.py` (lines 33–64):

```
def validate_session(self, session_id: str, device_fingerprint: str
    = None) -> Optional[User]:
     session_id_hash = hash_token(session_id)
     db_session =
        self.db.query(Session).filter(Session.session_id_hash ==
        session_id_hash).first()

     if not db_session:
         return None

     now = datetime.now(timezone.utc)

     # Absolute timeout check (2 hours)
     if db_session.expires_at.replace(tzinfo=timezone.utc) < now:
         self.delete_session(session_id)
         return None

     # Idle timeout check (10 minutes)
     idle_limit =
        db_session.last_activity.replace(tzinfo=timezone.utc) + \
         timedelta(minutes=settings.SESSION_IDLE_TIMEOUT)
     if idle_limit < now:
         self.delete_session(session_id)
         return None

     # Device fingerprint check
     if db_session.device_fingerprint and
        db_session.device_fingerprint != device_fingerprint:
         self.delete_session(session_id)
         return None

     # Update last activity
     db_session.last_activity = now
     self.db.commit()
     return db_session.user
```

9.3 Security Analysis

Strengths:

- Dual timeout mechanism provides layered protection: absolute timeout (2 hours) ensures sessions cannot persist indefinitely even if actively used, while idle timeout (10 minutes) quickly invalidates abandoned sessions
- Session rotation every 2.5 minutes reduces the hijacking window—even if a session ID is stolen, it becomes invalid shortly after
- Expired sessions are actively deleted on access, not just ignored
- `last_activity` timestamp updated on each valid request for accurate idle tracking

Weaknesses:

No Background Cleanup of Expired Sessions: Sessions are only deleted when accessed. If a user abandons a session (closes browser, never returns), the session record persists in the database until the expiry time passes AND someone attempts to use it.

Fix: Add cleanup job (scheduled task) to delete sessions where `expires_at < now()`.

9.4 Classification

✓ **Secure**

10 Session Invalidation on Logout

10.1 Requirement Importance

Logout must completely destroy session state on both client and server. If sessions persist after logout, attackers with stolen session tokens can continue accessing accounts. The logout process must be immediate and complete.

10.2 Location in Code

File: `auth_lib/api/main.py` (lines 88–99):

```python
@app.post("/logout")
def logout(
    response: Response,
    request: Request,
    session_service: SessionService =
        Depends(deps.get_session_service)
):
    session_id = request.cookies.get("id")
    if session_id:
        session_service.delete_session(session_id)

    response.delete_cookie("id")
    return {"message": "Logged out"}
```

File: `auth_lib/services/session_service.py` (lines 80–84):

```python
def delete_session(self, session_id: str):
    """Invalidates a single session."""
    session_id_hash = hash_token(session_id)
    self.db.query(Session).filter(Session.session_id_hash ==
        session_id_hash).delete()
    self.db.commit()
```

10.3 Security Analysis

Strengths:

- Session record is physically deleted from database (not just marked invalid), ensuring immediate and complete invalidation
- Cookie explicitly deleted from client response
- Subsequent requests with the old session ID will fail validation (hash not found in database)
- Security middleware applies `Cache-Control: no-store` to prevent caching of authenticated responses

Weaknesses:

Refresh Tokens Not Explicitly Invalidated on Single-Session Logout: The `delete_session()` method only removes the session record. If refresh tokens were issued for this session (JWT flow), they are not invalidated.

> Attack Scenario: User logs out. Attacker who stole the refresh token can still use it to obtain new access tokens.
>
> Impact: Logout does not fully terminate access if refresh tokens are in use.

Fix: Bind refresh tokens to sessions. Alternatively, invalidate all refresh tokens on logout. Note: `invalidate_all_user_sessions()` does handle this, but single-session logout does not.

10.4 Classification

✓ Secure

11 Session Invalidation on Password Change

11.1 Requirement Importance

When a password is changed, all existing sessions must be terminated to prevent continued access by attackers who may have compromised a session. This is critical during security incidents where the user suspects credential theft.

11.2 Location in Code

File: `auth_lib/services/session_service.py` (lines 86–90):

```
def invalidate_all_user_sessions(self, user_id: int):
    """Invalidates all sessions for a user (e.g., after password
       reset)."""
    self.db.query(Session).filter(Session.user_id ==
       user_id).delete()
    self.db.query(RefreshToken).filter(RefreshToken.user_id ==
       user_id).delete()
    self.db.commit()
```

File: `auth_lib/api/main.py` (lines 115–119):

```
@app.post("/password-reset/confirm")
def confirm_password_reset(...):
    user = auth_service.complete_password_reset(data.token,
       data.new_password, data.totp_code)
    session_service.invalidate_all_user_sessions(user.id)
    return {"message": "Password reset successful. All sessions
       invalidated."}
```

11.3 Security Analysis

Strengths:

- All sessions for the user are deleted (not just current) after password reset, ensuring complete session termination
- All refresh-tokens are also revoked, preventing the attacker from re-authenticating if refresh-token is stolen

Weaknesses:

No Authenticated Password Change Endpoint: Only password reset (via email token) exists. There is no `/password-change` endpoint for authenticated users.

Fix: Add `POST /password-change` endpoint.

`is_verified` Not Enforced for Password Reset: Unverified users can still reset their password and continue using the system without email verification.

> Attack Scenario: Attacker registers with victim's email, never verifies. Attacker uses the account. Later, real owner tries to register, fails (email taken).
>
> Impact: Email squatting (legitimate email owner cannot prove it).
>
> Fix: Require email verification before allowing password reset.

11.4 Classification

△ Partially Secure

12 Cookie / Token Storage Configuration

12.1 Requirement Importance

Insecure cookie settings expose session tokens to XSS (missing `HttpOnly` header), network interception (missing `Secure` header), and CSRF (missing `SameSite` header) attacks.

12.2 Location in Code

File: `auth_lib/api/main.py` (lines 77–84):

```
response.set_cookie(
    key="id",                    # Generic name
    value=session_id,
    httponly=True,               # No JavaScript access
    secure=True,                 # HTTPS only
    samesite="lax"               # CSRF protection
)
```

File: `auth_lib/middleware/security_middleware.py` (lines 52–59):

```
if new_session_id:
    response.set_cookie(
        key="id",
        value=new_session_id,
        httponly=True,
        secure=True,
        samesite="lax"
    )
```

12.3 Security Analysis

Strengths:

- `HttpOnly=True` prevents JavaScript access, mitigating XSS-based session theft. Even if an attacker injects malicious script, they cannot read the session cookie
- `Secure=True` ensures cookie is only sent over HTTPS, preventing interception on unencrypted connections
- `SameSite=Lax` protects against CSRF on POST, PUT, and DELETE requests
- Generic cookie name “id” (doesn’t reveal “SESSIONID” or framework-specific names) prevents fingerprinting

Weaknesses:

None identified.

12.4 Classification

✓ **Secure**

13 Protection Against Session Fixation

13.1 Requirement Importance

Session fixation allows attackers to set a known session ID on a victim's browser before login. After the victim authenticates, the attacker can use the pre-set session ID to access the authenticated session. Prevention requires generating a new session ID after successful authentication.

13.2 Location in Code

File: `auth_lib/api/main.py` (lines 71–76):

```
# 5. Create Session (new session ID generated after authentication)
session_id, _ = session_service.create_session(
    user.id,
    device_fingerprint=request.state.device_fingerprint
)

# 6. Set Secure Cookie
response.set_cookie(
    key="id",
    value=session_id,
    httponly=True,
    secure=True,
    samesite="lax"
)
```

File: `auth_lib/services/session_service.py` (lines 66–78):

```
def rotate_session(self, old_session_id: str) -> Optional[str]:
    """Rotates a session ID (every 2.5 minutes)."""
    old_hash = hash_token(old_session_id)
    db_session =
       self.db.query(Session).filter(Session.session_id_hash ==
       old_hash).first()

    if not db_session:
        return None

    new_session_id = generate_session_id()
    db_session.session_id_hash = hash_token(new_session_id)
    self.db.commit()
    return new_session_id
```

13.3 Security Analysis

Strengths:

- New session ID is always generated after successful authentication in `create_session()`
- Any pre-existing session ID (e.g., set by an attacker) is never promoted to an authenticated session
- Session rotation every 2.5 minutes further limits the window for session theft—even if an attacker observes a valid session ID, it becomes invalid within minutes
- Old session ID is immediately invalidated on rotation (hash is replaced)
- Device fingerprint binding provides additional detection of session theft from different contexts

Weaknesses:

None identified.

13.4 Classification

✓ **Secure**

14 Privilege Separation / Role Checking

14.1 Requirement Importance

Role-based access control (RBAC) ensures users can only access resources they are allowed to access. Missing role checks allow privilege escalation. For a cryptocurrency wallet, this is critical—unauthorized access could result in fund theft.

14.2 Location in Code

File: `auth_lib/models/models.py` (lines 7–21):

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True, nullable=False)
    hashed_password = Column(String, nullable=False)
    is_verified = Column(Boolean, default=False)
    created_at = Column(DateTime, default=lambda:
        datetime.now(timezone.utc))
    last_password_change = Column(DateTime, default=lambda:
        datetime.now(timezone.utc))
    # NO ROLE FIELD EXISTS
```

File: `auth_lib/api/deps.py` (lines 23–42):

```
async def get_current_user(
    request: Request,
    db: Session = Depends(get_db),
    session_service: SessionService = Depends(get_session_service)
):
    session_id = request.cookies.get("id")
    if not session_id:
        raise HTTPException(status_code=401, detail="Not
            authenticated")

    user = session_service.validate_session(session_id)
    if not user:
        raise HTTPException(status_code=401, detail="Session invalid
            or expired")

    return user  # No role check, no is_verified check
```

14.3 Security Analysis

Strengths:

- Basic authentication is enforced on protected endpoints via `get_current_user`
- Session validation is performed before granting access
- Email verification flag exists (`is_verified`) for future enforcement

Weaknesses:

No Role/Permission Model Exists: The User model has no `role` or `permissions` field at all. All authenticated users have identical privileges.

Attack Scenario: If Admin functionality is added later (e.g., `/admin/users/delete`) without RBAC, any authenticated user can access it.

Impact: Privilege escalation.

Fix: Add role field to user data.

`is_verified` Flag Not Enforced: Users can register, skip email verification, and fully use the system. The `is_verified` field exists but is never checked.

Attack Scenario: Attacker registers with victim's email (e.g., alice@example.com). Attacker never verifies but gains full account access. Real Alice tries to register later and finds the email is taken.

Impact: Email squatting (legitimate email owner cannot prove it).

Fix: Check `is_verified` in `get_current_user`.

No Resource-Level Access Control: No mechanism to ensure users can only access their own resources.

Attack Scenario: If `/wallet/{wallet_id}` endpoint is added without ownership check, User A could access User B's wallet by guessing IDs.

Impact: Privilege escalation.

Fix: Implement mechanism to ensure user ownership checks.

14.4 Classification

× **Missing**

15 Cryptographic Key Management

15.1 Requirement Importance

Cryptographic keys protect sensitive data. All encrypted data could be compromised if keys are not properly managed. For a cryptocurrency wallet, this is catastrophic.

15.2 Location in Code

File: `auth_lib/core/config.py` (lines 9–12):

```
# Security Keys (Should be set in environment)
# Using default values only for development; production MUST set these
SECRET_KEY: str = "y3f8v2n9m4c7x1z0l9k8j7h6g5f4d3s2"
ENCRYPTION_KEY_LATEST: str = "MTIzNDU2Nzg5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTI="
```

File: `auth_lib/core/security.py` (lines 37–56):

```
class EncryptionManager:
    """Handles encryption and decryption with key rotation support."""
    def __init__(self, keys: list[str]):
        # keys[0] is the primary (newest) key
        self.fernets = [Fernet(k.encode() if isinstance(k, str) else k) for k in keys]
        self.multi_fernet = MultiFernet(self.fernets)

    def encrypt(self, data: str) -> str:
        return self.multi_fernet.encrypt(data.encode()).decode()

    def decrypt(self, encrypted_data: str) -> str:
        return self.multi_fernet.decrypt(encrypted_data.encode()).decode()

# Global encryption manager initialized at module load
encryption_manager = EncryptionManager([settings.ENCRYPTION_KEY_LATEST])
```

15.3 Security Analysis

Strengths:

- Uses Fernet encryption (AES-128-CBC + HMAC-SHA256), providing both confidentiality and integrity
- MultiFernet architecture supports key rotation without breaking existing encrypted data
- Keys are loaded via pydantic-settings which supports environment variable overrides
- Comments explicitly warn against using defaults in production

Weaknesses:

Default Keys Hardcoded in Source Code: The default `SECRET_KEY` and `ENCRYPTION_KEY_LATEST` are committed to the repository. Any insider with access to the source can decrypt all data encrypted with defaults.

Attack Scenario: Insider attacker who has the source code can:

1. Decrypt all TOTP secrets from database
2. Generate valid TOTP codes for any user
3. Bypass MFA

Impact: Complete authentication bypass.

Fix: Set default keys via environment variables.

Default Encryption Key Is Predictable: The default `ENCRYPTION_KEY_LATEST` decodes from Base64 to `"12345678901234567890123456789012"`—an obviously weak test key.

Attack Scenario: An attacker might guess common test keys even without source code access.

Impact: Complete authentication bypass.

Fix: Use properly generated default keys.

15.4 Classification

× **Insecure!**